

Generative AI Developer

Module 1: Understanding the GenAI Landscape

LLM Fundamentals

- What is a Large Language Model (LLM)
- What is a token
- Context window and its limits
- Temperature and randomness
- Hallucination — what it is and why it happens
- Determinism vs creativity
- Base model vs instruction-tuned model vs chat model

Model Ecosystem

- OpenAI GPT-4o, GPT-4o mini
- Anthropic Claude 3.5 Sonnet, Claude 3 Haiku
- Google Gemini 1.5 Pro, Gemini Flash
- Mistral, Mixtral, Phi-3
- Cohere Command R+
- Closed-source vs open-source models

Model Benchmarks

- LMSYS Chatbot Arena
- MMLU
- HumanEval
- How to read a model card

Model Capabilities

- Text generation
- Code generation
- Vision (image understanding)
- Audio (speech-to-text, text-to-speech)
- Embeddings
- Function calling / tool use
- Document understanding
- Structured output generation

Key Model Parameters

- `temperature`
- `max_tokens`
- `top_p`

- `top_k`
 - `stop` sequences
 - `seed`
 - `frequency_penalty`
 - `presence_penalty`
 - `system`, `user`, `assistant` message roles
-

Module 2: Working with LLM APIs

API Fundamentals

- How REST API calls to LLMs work
- API key authentication
- The `messages` array structure
- Constructing a basic API request
- Reading and parsing API responses
- HTTP status codes from LLM APIs
- Rate limits — RPM and TPM
- Retry logic and exponential backoff
- Error types — rate limit, context exceeded, content policy, server error
- Timeout handling

Streaming Responses

- What streaming is
- Server-Sent Events (SSE)
- Implementing streaming in Python
- Handling partial responses
- Handling stream interruptions

Token Management

- Counting tokens before a call
- `tiktoken` for OpenAI
- Claude's token counter
- Estimating cost per request (input vs output tokens)
- Context window limits per model
- Truncation vs summarization vs chunking for long inputs
- Building a cost calculator

SDKs and Developer Tooling

- OpenAI Python SDK
- Anthropic Python SDK
- Google Generative AI SDK
- LiteLLM for unified multi-provider access

Multi-turn Conversations

- Maintaining conversation history
 - Appending user and assistant turns
 - Managing context length in long conversations
 - Summarization strategy
 - Sliding window strategy
-

Module 3: Prompt Engineering

Prompt Design Foundations

- Anatomy of a good prompt
- System prompt vs user prompt
- Being explicit vs implicit
- Positive instructions vs negative instructions
- Output format control — plain text, JSON, Markdown, XML, tables
- Using delimiters — ```, <tags>, ---
- Specifying output length

Core Prompting & Reasoning Techniques

- Zero-shot prompting
- One-shot prompting
- Few-shot prompting
- Instruction hierarchy
- Chain-of-Thought (CoT)
- Zero-shot CoT
- Few-shot CoT
- Tree-of-Thought (ToT)
- ReAct prompting
- Meta-prompting

Structured Output Prompting

- Prompting for valid JSON
- XML tags for structured sections
- Prompting for Markdown
- Forcing consistent output with few-shot examples
- Handling malformed JSON and retry strategies
- JSON mode (OpenAI)
- Structured outputs with schema
- Tool use for structured extraction

System Prompt Engineering

- Writing robust system prompts for production
- Defining persona, tone, scope, and limitations
- Injecting dynamic context at runtime
- Handling out-of-scope inputs

- Versioning prompts as code
- A/B testing prompts

Prompt Security

- Prompt injection — direct and indirect
 - Prompt leaking prevention
 - Jailbreaking — attack patterns and countermeasures
 - Input validation before passing to LLM
 - Output filtering
-

Module 4: Embeddings and Semantic Search

Understanding Embeddings

- What an embedding is
- Why embeddings capture semantic meaning
- Embedding space and clustering
- Sparse vectors (TF-IDF, BM25) vs dense vectors
- Use cases — search, clustering, classification, RAG

Embedding Models

- OpenAI `text-embedding-3-small` and `text-embedding-3-large`
- Cohere `embed-english-v3.0` and `embed-multilingual-v3.0`
- Google `text-embedding-004`
- sentence-transformers — `all-MiniLM-L6-v2`, `BAAI/bge-large-en`
- Code embeddings
- Image embeddings (CLIP)
- Embedding dimensions — 768, 1536, 3072

Similarity and Distance Metrics

- Cosine similarity
- Dot product similarity
- Euclidean (L2) distance
- Computing cosine similarity in Python
- Interpreting similarity scores

Building a Semantic Search System

- Embedding a document corpus
 - Storing embeddings as NumPy arrays
 - Embedding the user query
 - Top-K retrieval
 - Minimal semantic search with FAISS
 - Hybrid search — BM25 + vector search
-

Module 5: Vector Databases

Core Concepts

- Why relational DBs are insufficient for vectors
- Approximate Nearest Neighbor (ANN) search
- HNSW indexing algorithm
- IVF (Inverted File Index)
- Flat index — exact search
- Metadata filtering alongside vector search

ChromaDB

- Installation and setup
- Creating collections
- Adding documents with embeddings and metadata
- Querying with text or vectors
- Metadata filtering
- Persistent vs in-memory storage

Qdrant

- Installation — Docker and cloud
- Collections, points, and payloads
- Filtering with payload conditions
- Sparse + dense hybrid search
- Named vectors for multi-vector use cases

FAISS

- IndexFlatL2 and IndexFlatIP
- IndexIVFFlat
- IndexHNSWFlat
- Saving and loading indexes from disk
- GPU-accelerated FAISS

Module 6: Retrieval-Augmented Generation (RAG)

RAG Architecture

- The core problem RAG solves
- RAG vs fine-tuning vs in-context learning
- Indexing pipeline — ingest → parse → chunk → embed → store
- Query pipeline — query → embed → retrieve → rerank → generate
- Naive RAG vs Advanced RAG vs Modular RAG

Document Ingestion and Parsing

- Loading PDFs — PyMuPDF, pdfplumber, LlamaParse

- Loading Word documents — python-docx
- Loading HTML — BeautifulSoup, Unstructured
- Loading CSV and Excel — pandas
- Loading code files — tree-sitter
- OCR for scanned documents — Tesseract, AWS Textract
- Table extraction strategies

Text Chunking Strategies

- Fixed-size chunking
- Recursive character text splitter
- Sentence-aware chunking
- Semantic chunking
- Markdown-aware chunking
- Code-aware chunking
- Chunk size selection — 256, 512, 1024 tokens
- Chunk overlap — purpose and typical sizes
- Metadata preservation per chunk

Building the Indexing Pipeline

- End-to-end pipeline architecture
- Batch embedding for efficiency
- Deduplication of chunks
- Incremental indexing
- Metadata schema design
- Storing vector and full document text

Building the Retrieval Pipeline

- Embedding the user query
- Top-K retrieval and choosing K
- Similarity score thresholds
- Metadata pre-filtering
- Post-filtering retrieved results

Advanced Retrieval Techniques

- Multi-query retrieval
- Self-RAG
- Corrective RAG (CRAG)

Reranking

- Why reranking improves precision
- Cross-encoder vs bi-encoder
- Cohere Rerank API
- BGE Reranker (open-source)
- FlashRank — lightweight local reranker

- Reciprocal Rank Fusion (RRF)

Augmentation and Generation

- RAG prompt template structure
- Handling missing context gracefully
- Citing sources in generated answers
- Multi-document synthesis
- Grounding instructions to prevent hallucination

RAG Evaluation

- RAGAS framework
 - Faithfulness
 - Answer Relevancy
 - Context Recall
 - Context Precision
 - Building a golden QA dataset
 - TruLens evaluation framework
 - Chunking ablation studies
 - Retrieval@K metrics
-

Module 7: Function Calling and Tool Use

Core Concepts

- What function calling is
- How it works — tool definition → model decision → execution → response
- OpenAI function calling specification
- Anthropic tool use specification
- Gemini function calling
- Function calling vs RAG vs plain prompting

Defining Tools

- JSON schema for tool definitions
- **name**, **description**, **parameters** fields
- Writing effective tool descriptions
- Required vs optional parameters
- Enum types for constrained inputs
- Nested object schemas
- Arrays as parameter types

Executing Tool Calls

- Detecting when the model calls a tool
- Extracting tool name and arguments
- Executing the tool in code

- Returning tool results to the model
- Completing the conversation with the final response
- Multi-turn tool use

Parallel Tool Calling

- What parallel tool calling is
- OpenAI parallel tool calls implementation
- Executing multiple tools concurrently
- Aggregating results from parallel calls

Building Custom Tools

- Web search tool — Tavily, SerpAPI, DuckDuckGo
- Calculator / code execution tool
- Database query tool
- REST API caller tool
- File reader / writer tool
- Email sender tool
- Calendar event creator tool
- Browser / web scraper tool

Tool Use Patterns

- Tool selection via descriptions
 - Tool chaining
 - Error handling when a tool fails
 - Tool result formatting
 - Role-based tool access control
-

Module 8: Orchestration Frameworks

- LangChain
 - LlamaIndex
 - CrewAI
 - Claude Coding Agentic Framework
-

Module 9: Single Agent Systems

Core Concepts

- What is an AI agent
- Agent vs a simple LLM call vs a chain
- The agent loop — Perceive → Think → Act → Observe
- When to use an agent vs a fixed pipeline
- Limitations of single agents — reliability, latency, cost

Agent Architecture

- Components of an agent — LLM brain, tools, memory, executor
- System prompt design for agents
- Tool registry — how agents discover and select tools
- Scratchpad — how agents track intermediate reasoning
- Stop conditions — when an agent decides it is done

ReAct Agent Pattern

- Reasoning and Acting interleaved
- Thought → Action → Observation loop
- Implementing ReAct from scratch
- ReAct with LangChain AgentExecutor
- ReAct with LlamaIndex ReActAgent

OpenAI Tools Agent

- How OpenAI function calling powers agents
- `create_tool_calling_agent` in LangChain
- `OpenAIAgent` in LlamaIndex
- Structured tool outputs for agent decisions

LangGraph Single Agent

- Modeling an agent as a graph
- State schema for agent scratchpad
- Node — call LLM
- Node — execute tool
- Conditional edge — continue loop or end
- Compiling and running the agent graph

Agent Memory

- Short-term memory — conversation history in context
- Injecting tool results back into context
- Carrying state across agent steps
- Connecting agent to a long-term memory store

Error Handling and Reliability

- Max iteration limit to prevent infinite loops
- Handling tool execution failures gracefully
- Fallback behavior when LLM produces invalid tool calls
- Logging every agent step for debugging
- Timeout on individual tool calls

Agent Evaluation

- Measuring task completion rate
- Trajectory evaluation — were the right tools called in the right order

- Step efficiency — did the agent reach the answer in minimum steps
 - Human review of agent traces in LangSmith
-

Module 10: Multi-Agent Systems

Core Concepts

- Why multi-agent over single-agent
- Task specialization
- Parallel execution across agents
- Verification and self-correction
- Scalability

Agent Roles

- Planner agent
- Executor agent
- Critic / Reviewer agent
- Researcher agent
- Writer / Generator agent
- Orchestrator / Supervisor agent

Communication Patterns

- Hub-and-spoke
- Peer-to-peer
- Hierarchical
- Blackboard (shared state)
- Pipeline (linear handoff)

Task Decomposition

- Breaking a goal into sub-tasks
- Dependency graphs between tasks
- Parallel vs sequential execution
- Merging results from multiple agents

Reliability Patterns

- Retry logic for failed agent steps
- Timeouts on tool calls
- Fallback agents
- Guardrails on agent outputs
- Max iteration limits
- Logging every agent step

Human-in-the-Loop

- When to pause for human approval

- Interrupting a LangGraph workflow
 - Presenting plan before execution
 - Accepting, rejecting, or editing agent steps
 - Audit trail of agent decisions
-

Module 11: Memory Systems

Types of Memory

- In-context (short-term) memory
- External (long-term) memory
- Episodic memory
- Semantic memory
- Procedural memory

Short-Term Memory Management

- Conversation history as messages list
- Buffer window — last N turns
- Summarization of old turns
- Token-aware truncation
- Summary buffer — hybrid approach

Long-Term Memory with Vector Stores

- Storing user facts as embeddings
- Storing past conversation summaries
- Retrieving relevant memories at query time
- Memory consolidation
- Memory expiry and freshness

External State Stores

- Redis for session state
- PostgreSQL for structured memory
- SQLite for local persistence
- LangGraph checkpointers

Memory in Practice

- Injecting retrieved memories into system prompt
 - User profile memory across sessions
 - Project context memory
 - Mem0 — open-source memory layer
 - Zep — LLM-specific memory store
-

Module 12: Building Production APIs

API Design for GenAI

- Sync vs async endpoints
- Streaming endpoints
- WebSocket endpoints
- Background tasks and status polling
- REST vs WebSocket vs SSE

FastAPI for GenAI Backends

- Project structure setup
- Async route handlers
- Pydantic request/response models
- Dependency injection
- StreamingResponse
- WebSocket endpoint
- BackgroundTasks
- CORS, auth, rate limiting middleware
- Exception handlers for LLM errors

Session and State Management

- Stateless API design
- Redis for server-side session storage
- Conversation ID with UUID
- Multi-tenant session isolation
- Session expiry and cleanup

File Handling

- Accepting file uploads via multipart form
- File type and size validation
- Async file processing pipelines
- Temporary file storage and cleanup

Module 13: Observability and Monitoring

Why Observability Matters

- Non-deterministic outputs need visibility
- Debugging prompt issues
- Cost visibility per user and feature
- Quality degradation detection
- Latency monitoring

LangSmith

- Setting up tracing

- Automatic tracing with LangChain
- Manual tracing for non-LangChain code
- Trace explorer
- Inspecting inputs, outputs, intermediate steps
- Adding metadata to runs
- Creating datasets and running evaluations
- Comparing prompt versions with experiments

Langfuse

- Self-hosting vs Langfuse Cloud
- Tracing via Python SDK decorator
- Manual tracing with spans and generations
- User and session tracking
- Cost tracking per model
- Dashboards — latency, cost, error rate
- Prompt management

Metrics to Track

- Latency — TTFT and total generation time
- Token usage — input, output, total
- Cost per request, per user, per feature
- Error rate — API errors, tool failures, timeouts
- Cache hit rate
- Retrieval relevance scores
- User satisfaction scores

Logging Best Practices

- What to include in prompt/response logs
- PII scrubbing before logging
- Structured JSON logging
- Correlation IDs across services
- Log levels — DEBUG, INFO, ERROR
- Log storage — Elasticsearch, Loki, CloudWatch

Module 14: Caching and Cost Optimization

Why Caching is Critical

- LLM calls are expensive and slow
- Semantically similar queries are common
- Caching reduces costs by 30–70% in production

Exact-Match Caching

- Caching by exact prompt hash

- Redis for fast lookup
- TTL settings
- Cache invalidation strategies

Semantic Caching

- How semantic caching works
- GPTCache — open-source semantic cache
- Redis with vector extension
- Similarity threshold tuning
- Cache hit rate measurement

LiteLLM

- Unified interface to 100+ providers
- Automatic model routing
- Budget management per project/user
- Fallback chains across models
- Load balancing across API keys
- Cost tracking and reporting

Prompt Compression

- LLMingua
- LongLLMingua
- Extractive compression
- Abstractive compression
- Compression ratio vs quality tradeoff

Model Routing Strategy

- Cheap models for simple tasks
- Powerful models for complex tasks
- Classifying task complexity before routing
- Cascading — small model first, escalate if needed

Module 15: Guardrails and Safety

Why Safety Matters

- Harmful and biased content risks
- Business and legal risk
- User trust and reliability
- Compliance — GDPR, HIPAA, SOC2

Input Validation

- Detecting prompt injection in user input
- Blocking disallowed topics

- Input length limits
- Character encoding sanitization
- Language detection and filtering

Output Validation

- Rule-based output checks
- JSON schema validation
- Content policy checks
- Faithfulness check against context
- Regex and rule-based post-processing

Guardrails AI

- NL → validation rule pipeline
- Built-in validators — toxicity, PII, URL presence
- Custom validators
- Fixing invalid outputs — reask strategy
- On-fail actions — reask, filter, exception, noop

NeMo Guardrails

- Colang language for defining rails
- Input rails
- Output rails
- Dialog rails
- LangChain integration

PII Detection and Redaction

- Types of PII — names, emails, phone, SSN, credit cards
- Microsoft Presidio
- Redacting PII before LLM
- Reversing anonymization in the response
- Scrubbing PII from logs

Content Moderation

- OpenAI Moderation API
- Llama Guard
- Perspective API
- Building a custom moderation classifier
- Moderation on input and output

Module 16: LLM Evaluation

Why Evaluation Matters

- Non-deterministic outputs require systematic quality measurement

- Catching regressions when prompts or models change
- Building confidence before shipping to production
- Measuring quality across diverse and adversarial inputs

Evaluation Fundamentals

- Reference-based vs reference-free evaluation
- Automated vs human evaluation
- Offline evaluation vs online (live) evaluation
- Unit tests vs regression suites vs benchmark suites
- The eval-driven development workflow

Building Evaluation Datasets

- What makes a good eval dataset
- Collecting seed examples from real usage
- Synthetic data generation for edge cases
- Golden dataset — expected inputs and outputs
- Adversarial examples — jailbreaks, ambiguous queries, out-of-scope inputs
- Dataset versioning and management

Metrics for Text Generation

- Exact match
- BLEU score
- ROUGE-1, ROUGE-2, ROUGE-L
- BERTScore — semantic similarity with embeddings
- METEOR
- Perplexity
- When each metric is appropriate

LLM-as-a-Judge

- Using a strong LLM to score another LLM's output
- Pointwise scoring — rating a single response
- Pairwise comparison — choosing the better of two responses
- Writing effective judge prompts
- Bias in LLM judges — position bias, verbosity bias, self-preference
- Calibrating judge scores against human labels

Task-Specific Evaluation

- Summarization — faithfulness, coverage, conciseness
- Question answering — accuracy, groundedness, hallucination rate
- Code generation — functional correctness, execution pass rate
- Classification — precision, recall, F1
- Dialogue — coherence, engagingness, task completion
- Instruction following — format compliance, constraint adherence

Hallucination Detection

- Types of hallucination — factual, faithfulness, reasoning
- Entailment-based detection
- SelfCheckGPT — sampling-based consistency check
- FactScore — claim-level factuality scoring
- Chainpoll — multi-sample polling for hallucination

Evaluation Frameworks

- OpenAI Evals
- EleutherAI LM Evaluation Harness
- DeepEval
- Promptfoo
- Braintrust
- Weights & Biases Weave

Human Evaluation

- When human eval is necessary
- Designing annotation guidelines
- Inter-annotator agreement — Cohen's Kappa, Krippendorff's Alpha
- Labeling platforms — Scale AI, Labelbox, Argilla
- Sampling strategies for cost-efficient human review
- Preference data collection for RLHF

Regression Testing and CI/CD

- Running evals in CI pipelines
- Thresholds and pass/fail gates
- Tracking metric trends over time
- Alerting on quality regressions
- Prompt versioning tied to eval results

Module 17: Capstone Project